

HTML

CHEAT SHEETS

A Complete Reference for Modern HTML

Document Structure

Semantics

Forms

Media

Accessibility

Links

Tables

DOM & Script

Typography

Modern HTML

15 REFERENCE SHEETS

CONTENTS

FOUNDATIONS & STRUCTURE

- 01 Architecture & Organization
- 02 Head & Metadata
- 03 Landmark & Structural Tags
- 04 Semantic Containers
- 05 Generic Containers

CONTENT & MEDIA

- 06 Classes & IDs
- 07 Typography
- 08 Lists
- 09 Tables
- 10 Inline & Misc Elements

INTERACTIVE & ADVANCED

- 11 Media
- 12 Links
- 13 Forms
- 14 Accessibility
- 15 DOM & Script Basics

Covers [HTML5](#) · [Semantic Elements](#) · [ARIA](#) · [Forms](#) · [Media](#) · [Modern HTML \(2024\)](#)

[Begin](#) → [Sheet 01](#)

Sheet 1

Architecture

document structure · elements · attributes · entities · parsing

Sheet 2

Head

metadata · SEO · open graph · favicons · CSS & JS loading

Sheet 3

Landmark & Structural Tags

header · nav · main · footer · aside · ARIA landmarks

Sheet 4

Semantic Containers

section · article · figure · time · address · blockquote

Sheet 5

Generic Containers

div · span · block vs inline · data-* attributes

Sheet 6

Classes & IDs

class · id · CSS selectors · naming · specificity

Sheet 7

Typography

headings · paragraphs · emphasis · code · inline text elements

Sheet 8

Lists

ul · ol · li · dl · dt · dd · nested lists · nav pattern

Sheet 9

Tables

table · thead · tbody · tfoot · th · td · caption · scope · colspan

Sheet 10

Inline & Misc Elements

br · hr · wbr · details · summary · dialog · template · noscript

Sheet 11

Media

img · srcset · picture · video · audio · iframe · canvas · SVG

Sheet 12

Links

href · absolute · relative · target · rel · schemes · download · states

Sheet 13

Forms

form · input types · textarea · select · button · label · fieldset · validation

Sheet 14

Accessibility

ARIA roles · states · properties · keyboard · focus · live regions

Sheet 15

DOM & Script Basics

DOM · script loading · querySelector · innerHTML · dataset · events

DESCRIPTION

HTML (HyperText Markup Language) is the skeleton of every web page — a text file of *elements* that tell the browser what content exists and what it means, not how it looks. A browser reads the HTML file top-to-bottom, builds a tree of objects called the **DOM** (Document Object Model), and renders that tree to the screen. CSS handles appearance; JavaScript handles behavior; HTML handles *structure and meaning*.

EXAMPLE

```
<!-- ① DOCTYPE: tells the browser this is modern HTML5 -->
<!DOCTYPE html>

<!-- ② Root element: lang sets the document language -->
<html lang="en">

  <!-- ③ Head: metadata — nothing here renders visibly -->
  <head>
    <meta charset="UTF-8" />           <!-- character encoding -->
    <meta name="viewport"
      content="width=device-width,
        initial-scale=1.0" /> <!-- mobile scaling -->
    <title>Page Title</title>         <!-- browser tab text -->
  </head>

  <!-- ④ Body: everything visible goes here -->
  <body>

    <!-- Container element: opening tag + content + closing tag -->
    <p class="intro" id="lead">Hello, world.</p>
    <!--   ↑ attribute   ↑ attribute value   ↑ content   -->

    <!-- Void element: no closing tag, no children -->
    

    <!-- Boolean attribute: presence = true, no value needed -->
    <input type="checkbox" checked />

    <!-- Custom data attribute: store data for CSS / JS -->
    <button data-action="save">Save</button>

    <!-- Character entities: use when char has HTML meaning -->
    <p>Use &lt;div> not &lt;span> for block layout.</p>
    <!-- renders: Use <div> not <span> for block layout. -->

    <!-- Whitespace: multiple spaces / newlines collapse to one -->
    <p>Word   word   word</p>
    <!-- renders: Word word word -->

  </body>
</html>
```

NOTES

- HTML is **not** a programming language — it has no logic, loops, or variables
- Browsers never throw errors on bad HTML — they silently repair and continue
- File extension is `.html` (or `.htm` — legacy, avoid)
- HTML is case-insensitive, but lowercase tags and attributes are the standard
- The current standard is **HTML5** (living standard, maintained by WHATWG)

ELEMENT TYPES

TYPE	DESCRIPTION
Container	Has opening + closing tag; wraps content — <code><p></code> , <code><div></code> , <code><a></code>
Void	No closing tag; no children — <code></code> , <code>
</code> , <code><input></code> , <code><meta></code>
Block	Starts a new line, full width — <code><p></code> , <code><div></code> , <code><h1></code>
Inline	Flows within text — <code></code> , <code><a></code> , <code></code>
Comment	Ignored by browser — <code><!-- text --></code>

COMMON CHARACTER ENTITIES

ENTITY	CHAR	WHEN TO USE
<	<	Less-than sign in text
>	>	Greater-than sign in text
&	&	Ampersand in text or attr
"	"	Quote inside an attribute
 	␣	Non-breaking space
©	©	Copyright symbol
—	—	Em dash

ARCHITECTURAL TAGS

TAG	DESCRIPTION
<!DOCTYPE html>	Declares HTML5; must be the very first line — no content before it
<html>	Root element; lang attribute is required for ally
<head>	Metadata container — nothing here renders visibly on the page
<body>	All visible page content goes here; only one allowed per document
<title>	Browser tab text; required — affects SEO and ally
<meta>	Machine-readable metadata: charset, viewport, description
<link>	Links external resources (stylesheets, icons) into the document
<script>	Embeds or links JavaScript; use defer to avoid blocking parse

3 ACCESSIBILITY

- Semantic elements give screen readers free **role**, **name**, and **state** — always prefer them over `<div>`
- The `lang` attribute on `<html>` tells screen readers which language to pronounce content in — always include it
- Document reading order follows DOM order — structure your HTML in the order a screen reader should encounter it
- A well-structured HTML document is the single highest-impact ally investment

⚠ COMMON MISTAKES

- Omitting `<!DOCTYPE html>` triggers quirks mode — browsers render inconsistently
- Forgetting the closing tag on container elements — browser repairs it, but not always where you expect
- Using `&` bare in text or URLs inside attributes — must be `&`
- Duplicate `id` values on a page — ids must be unique; duplicates break CSS, JS, and ally
- Omitting `lang` on `<html>` — screen readers guess the language, often wrongly

✓ TIPS & BEST PRACTICES

- Always start with the HTML5 boilerplate — `DOCTYPE`, `lang`, `charset`, `viewport`, `title`
- Use lowercase for all tag names and attribute names — it's the universal convention
- Quote all attribute values with double quotes, even when technically optional
- Use the W3C validator (`validator.w3.org`) to catch structural errors the browser silently hides
- Indent nested elements consistently — structure mirrors the DOM tree

QUICK SUMMARY

- HTML = structure & meaning; CSS = appearance; JS = behavior
- `<!DOCTYPE html>` must be the very first line
- Elements are either *container* (with closing tag) or *void* (no closing tag)
- Attributes live in the opening tag: `name="value"`
- Browsers silently fix bad HTML — use a validator to surface real errors

DESCRIPTION

The `<head>` element contains metadata about the document — information the browser and search engines need but that users never see directly. It controls character encoding, viewport scaling, the browser tab title, SEO data, social sharing previews, icons, and how external CSS and JavaScript files are loaded. Getting the `<head>` right is foundational: errors here affect rendering, performance, SEO, and accessibility before a single visible element appears.

NOTES

- `charset` must appear within the first 1024 bytes of the file — put it first
- `viewport` meta is required for responsive design — without it, mobile browsers zoom out
- `<title>` is the single most important SEO element on the page
- Open Graph tags are read by social platforms (Facebook, Slack, LinkedIn) when generating link previews
- `defer` and `async` only work on external scripts — inline `<script>` blocks are always render-blocking

SCRIPT LOADING STRATEGIES

ATTRIBUTE	DOWNLOADS	EXECUTES
<code>none</code>	Blocks parsing	Immediately, blocks render
<code>defer</code>	Parallel	After HTML parsed, in order
<code>async</code>	Parallel	As soon as downloaded
<code>type="module"</code>	Parallel	Deferred, ES module scope

RESOURCE HINTS

REL VALUE	WHAT IT DOES
<code>preload</code>	Fetch this resource now, high priority (fonts, hero images)
<code>prefetch</code>	Fetch for likely next navigation, low priority
<code>preconnect</code>	Open TCP/TLS connection to origin early
<code>dns-prefetch</code>	Resolve DNS for origin early, lighter than preconnect

3 ACCESSIBILITY

- `<title>` is announced by screen readers when the page loads — make it unique and descriptive per page
- `lang` on `<html>` (set in Architecture) enables correct pronunciation by screen readers — the head does not override it
- Use `<meta name="theme-color">` carefully — ensure sufficient contrast in browser UI chrome for users with low vision
- Avoid `<meta http-equiv="refresh">` auto-redirects — they disorient screen reader and keyboard users

⚠ COMMON MISTAKES

- Placing `<script>` without `defer` or `async` in `<head>` — blocks HTML parsing entirely
- Missing or generic `<title>` — "Home" or "Untitled" hurts SEO and confuses users with multiple tabs
- Omitting `og:image` — social previews show a blank card instead of your image
- Using `async` for scripts that depend on each other — execution order is not guaranteed
- Linking CSS at the bottom of `<body>` — causes a flash of unstyled content (FOUC)

EXAMPLE

```
<head>

<!-- ① Encoding: must be first, within 1024 bytes -->
<meta charset="UTF-8" />

<!-- ② Viewport: required for responsive layouts -->
<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<!-- ③ Title: shown in tab, bookmarks, search results -->
<title>About Us — Acme Corp</title>

<!-- ④ SEO meta -->
<meta name="description" content="Acme Corp makes everything." />
<meta name="robots" content="index, follow" />
<link rel="canonical" href="https://acme.com/about" />

<!-- ⑤ Open Graph — social share previews -->
<meta property="og:title" content="About Us — Acme Corp" />
<meta property="og:description" content="Acme Corp makes everything." />
<meta property="og:image" content="https://acme.com/og.jpg" />
<meta property="og:url" content="https://acme.com/about" />
<meta property="og:type" content="website" />

<!-- ⑥ Twitter / X card -->
<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:title" content="About Us — Acme Corp" />

<!-- ⑦ Favicon -->
<link rel="icon" href="/favicon.svg" type="image/svg+xml" />
<link rel="icon" href="/favicon.ico" sizes="any" /> <!-- fallback -->
<link rel="apple-touch-icon" href="/apple-touch-icon.png" />
<link rel="manifest" href="/site.webmanifest" />

<!-- ⑧ CSS: link in head so it loads before body renders -->
<link rel="stylesheet" href="/css/main.css" />

<!-- ⑨ Preconnect to a third-party origin (e.g. Google Fonts) -->
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />

<!-- ⑩ Preload a critical asset (hero font) -->
<link rel="preload" href="/fonts/inter.woff2"
      as="font" type="font/woff2" crossorigin />

<!-- ⑪ JavaScript: defer keeps it non-blocking -->
<script src="/js/app.js" defer></script>

<!-- ⑫ Theme color: browser chrome accent on mobile -->
<meta name="theme-color" content="#1a1a2e" />

</head>
```

✓ TIPS & BEST PRACTICES

- Order: `charset` → `viewport` → `title` → meta → CSS → preloads → scripts
- Keep `<title>` under 60 characters — search engines truncate longer titles
- Use `defer` for almost all scripts — it's parallel download + correct execution order
- Use an SVG favicon — it scales to any size and supports dark mode via `prefers-color-scheme`
- Validate Open Graph with the Facebook Sharing Debugger or [opengraph.xyz](#) before launch

QUICK SUMMARY

- `charset` + `viewport` + `title` are the three non-negotiable head elements
- CSS goes in `<head>`; scripts get `defer` or go before `</body>`
- Open Graph tags control how links look when shared on social platforms
- Resource hints (`preload`, `preconnect`) are performance tools, not required

DESCRIPTION

Landmark elements define the major navigable regions of a page. Unlike generic `<div>`s, each one carries an implicit ARIA landmark role that allows screen reader users to jump directly between page regions (banner, navigation, main content, footer) using keyboard shortcuts. They also signal page structure to search engines. A page without landmark elements forces screen reader users to read every line from the top — a significant usability failure.

EXAMPLE

```
<!-- Skip link — must be the first focusable element on the page -->
<a href="#main-content" class="skip-link">Skip to content</a>

<!-- banner landmark (page-level header) -->
<header>
  <!-- site logo, branding -->

  <!-- navigation landmark — label required when multiple navs exist -->
  <nav aria-label="Primary">
    <!-- primary nav links (ul > li > a) -->
  </nav>
</header>

<!-- main landmark — one per page, the dominant content -->
<main id="main-content">

  <!-- complementary landmark — sidebar, callout, related content -->
  <aside aria-label="Filters">
    <!-- filter controls or supplementary content -->
  </aside>

  <!-- primary page content: articles, sections, media, etc. -->

</main>

<!-- second nav — different label so screen readers can distinguish -->
<nav aria-label="Breadcrumb">
  <ol>
    <!-- breadcrumb trail links -->
  </ol>
</nav>

<!-- contentinfo landmark (page-level footer) -->
<footer>
  <!-- copyright, legal links, footer nav -->
  <nav aria-label="Footer">
    <!-- footer links -->
  </nav>
</footer>
```

COMMON MISTAKES

- Using `<div class="header">` instead of `<header>` — loses the landmark role entirely
- Nesting `<main>` inside `<header>` or `<footer>` — `<main>` must be a direct child of `<body>`
- Multiple unlabelled `<nav>` elements — screen readers list them as "navigation, navigation, navigation"
- Putting the skip link after other focusable elements — it must be the very first focusable item
- Wrapping all content in `<section>` — sections without headings provide no landmark benefit

NOTES

- Only one `<main>` per page — it marks the primary, unique content
- `<header>` and `<footer>` can appear inside `<article>` or `<section>` as well as at page level
- Multiple `<nav>` elements are fine — label them with `aria-label` to distinguish (e.g. "Primary", "Breadcrumb")
- These are CSS-neutral — they render as block elements with no default visual styling
- `<body>` itself is a landmark (document) — you never need `role="document"`

LANDMARK ELEMENTS & THEIR ARIA ROLES

ELEMENT	IMPLICIT ROLE	PURPOSE
<code><header></code>	banner (<i>page-level</i>)	Site header, logo, site-wide nav
<code><nav></code>	navigation	Major navigation link groups
<code><main></code>	main	Dominant page content — one per page
<code><footer></code>	contentinfo (<i>page-level</i>)	Site footer, copyright, links
<code><aside></code>	complementary	Tangential content — sidebars, callouts
<code><section></code>	region (<i>if named</i>)	Thematic grouping with a heading
<code><form></code>	form (<i>if named</i>)	Interactive form landmark

NESTING RULES

RULE	DETAIL
One <code><main></code>	Never nest or repeat; hide others with hidden in SPAs
<code><main></code> not in <code><header></code>	<code><main></code> is a sibling of <code><header></code> , not a child
Label multiple navs	Add <code>aria-label="Primary"</code> when more than one <code><nav></code> exists
header/footer context	Only get banner/contentinfo roles when direct child of <code><body></code>

ACCESSIBILITY

- Screen reader users navigate by landmark with a single keystroke — landmarks replace the need to tab through everything
- Add a "Skip to main content" link as the first focusable element: `<a href="#main">Skip to content</a>`
- When two landmarks share the same role (e.g. two `<nav>`), add `aria-label` to both to distinguish them in the landmark list
- Avoid landmark overuse — wrapping every `<div>` in a `<section>` clutters the landmark tree

TIPS & BEST PRACTICES

- Build the landmark skeleton first, before any inner content — it enforces good document structure
- Test landmark navigation with a screen reader (NVDA, VoiceOver) or the Accessibility Tree in DevTools
- Every page should have at least: `<header>`, `<main>`, `<footer>`
- Use `aria-label` on landmarks only when two or more share the same role

QUICK SUMMARY

- Landmark elements expose page regions to screen readers — use them instead of `<div>`
- One `<main>` per page; multiple `<nav>` / `<header>` / `<footer>` are allowed
- Label duplicate landmarks with `aria-label`
- Always include a skip link targeting `<main>` as the first focusable element

DESCRIPTION

Semantic containers go beyond landmarks — they describe the *nature* of their content rather than just its position on the page. An `<article>` is self-contained and syndicable (a blog post, a tweet, a product card). A `<section>` is a thematic grouping that needs a heading. `<figure>` wraps any self-contained illustration. These elements tell browsers, search engines, and assistive technology what the content *is*, not just where it sits — enabling richer indexing, reading modes, and accessible navigation.

NOTES

- `<section>` should always contain a heading (`<h2>` – `<h6>`) — a section without one is likely a `<div>`
- `<article>` can be nested — a comment inside a blog post is an article inside an article
- `<time datetime>` value must be a valid ISO 8601 format: `2025-01-15` , `14:30` , `2025-01-15T14:30:00`
- `<address>` is for contact info of the nearest `<article>` or `<body>` ancestor — not for postal addresses in general
- `<figure>` content can be an image, code block, chart, table, or video — anything self-contained with a caption

ELEMENT DECISION GUIDE

ELEMENT	USE WHEN...
<code><article></code>	Content makes sense on its own — post, card, comment, widget
<code><section></code>	Thematic grouping within a page or article; needs a heading
<code><figure></code>	Self-contained media or illustration referenced from main content
<code><figcaption></code>	Label or description for a <code><figure></code> ; first or last child only
<code><time></code>	Any date, time, or duration; add <code>datetime</code> for machine readability
<code><address></code>	Contact info for the document or nearest article author
<code><blockquote></code>	Extended quotation from another source; add <code>cite</code> attr for URL
<code><cite></code>	Title of a referenced work (book, film, article) — not the author name

ACCESSIBILITY

- `<article>` has an implicit role of `article` — screen readers announce "article" when entering, helping users understand content boundaries
- `<section>` only becomes a named landmark region when it has an `aria-label` or `aria-labelledby` — otherwise it's invisible to landmark navigation
- `<figure>` with `<figcaption>` is automatically associated — no extra ARIA needed; screen readers announce the caption as the figure's label
- `<time datetime>` lets assistive tech present dates in user-preferred formats — always include the `datetime` attribute

COMMON MISTAKES

- Using `<section>` as a styled wrapper with no heading — that's a job for `<div>`
- Using `<address>` for a physical mailing address unrelated to the author or document
- Omitting the `datetime` attribute from `<time>` — the human-readable text alone isn't machine-parseable
- Putting `<figcaption>` in the middle of a `<figure>` — it must be the first or last child
- Using `<cite>` for author names — it's for titles of works, not people

TIPS & BEST PRACTICES

- Ask "could this content be copy-pasted into an RSS feed and still make sense?" — if yes, it's an `<article>`
- Ask "does this group need its own heading?" — if yes, it's a `<section>`; if no, it's a `<div>`
- Always include `datetime` on `<time>` even when the visible text is already a date
- Use `<figure>` for any illustration the prose references by number — "see Fig. 1"

QUICK SUMMARY

- `<article>` = self-contained, syndicable;
- `<section>` = thematic group with heading
- `<figure>` + `<figcaption>` — any self-contained media with a label
- `<time datetime="ISO">` — always include the machine-readable attribute
- If you're unsure between `<section>` and `<div>`, ask: does it need a heading? If not, use `<div>`

EXAMPLE

```
<!-- article: self-contained, could be syndicated independently -->
<article>
  <header>
    <h2>How to Grow Tomatoes</h2>

    <!-- time: human-readable text + machine-readable datetime -->
    <p>Published <time datetime="2025-06-01">June 1, 2025</time></p>

    <!-- address: contact info for this article's author -->
    <address>
      By <a href="/authors/jane">Jane Smith</a>
    </address>
  </header>

  <!-- section: thematic grouping — always needs its own heading -->
  <section>
    <h3>Choosing a Location</h3>
    <p><!-- section body content --></p>

    <!-- figure: self-contained illustration referenced from prose -->
    <figure>
      
      <!-- figcaption must be first or last child of figure -->
      <figcaption>Fig. 1 — Ideal sun exposure for a south-facing bed.</figcaption>
    </figure>
  </section>

  <section>
    <h3>Expert Advice</h3>

    <!-- blockquote: extended quote; cite attr holds the source URL -->
    <blockquote cite="https://garden.org/tomatoes">
      <p><!-- quoted text --></p>
      <!-- cite element: title of the referenced work (not the author) -->
      <footer>— <cite>National Gardening Association Guide</cite></footer>
    </blockquote>
  </section>

  <!-- figure can wrap code blocks, tables, or any self-contained media -->
  <figure>
    <pre><code><!-- code snippet --></code></pre>
    <figcaption>A twice-weekly watering schedule.</figcaption>
  </figure>

  <!-- nested article: a comment is its own article inside the post -->
  <section aria-label="Comments">
    <h3>Comments</h3>
    <article>
      <p><strong>GardenFan99</strong> — Great tips, thanks!</p>
      <time datetime="2025-06-03T09:15">June 3 at 9:15 AM</time>
    </article>
  </section>
</article>
```

DESCRIPTION

`<div>` and `<span>` are the workhorses of HTML layout — generic containers with no semantic meaning and no default styling beyond their display type. `<div>` is block-level (starts a new line, full available width) and is used for layout wrappers, card shells, and CSS/JS hooks. `<span>` is inline (flows within text) and is used to target a portion of text for styling or scripting. Use them when no semantic element fits — always prefer a semantic element first.

NOTES

- `<div>` and `<span>` carry zero semantic meaning — they provide no role to screen readers
- Block elements stack vertically; inline elements flow horizontally within a line
- `data-*` attributes let you store custom data on any element — accessed in JS via `element.dataset`
- CSS can change any element's display type (`display: block` on a `<span>`) — but this doesn't change its semantic meaning
- Nesting block elements inside inline elements is invalid HTML — e.g. `<span><div>...</div></span>` is wrong

BLOCK VS INLINE ELEMENTS		
PROPERTY	BLOCK	INLINE
Default display	Starts on new line	Flows in text
Default width	Full parent width	Fits content
Can contain	Block + inline	Inline only
Examples	<code><div></code> <code><p></code> <code><h1></code>	<code></code> <code><a></code> <code></code>
Generic element	<code><div></code>	<code></code>

DATA-* ATTRIBUTE API	
HTML ATTRIBUTE	JS ACCESS VIA DATASET
<code>data-user-id="42"</code>	<code>el.dataset.userId</code> → "42"
<code>data-theme="dark"</code>	<code>el.dataset.theme</code> → "dark"
<code>data-max-count="10"</code>	<code>el.dataset.maxCount</code> → "10"
CSS targeting	<code>[data-theme="dark"] { ... }</code>

ACCESSIBILITY

- `<div>` and `<span>` are invisible to screen readers as structural elements — add `role` + `aria-label` only if making them interactive
- Never use a `<div>` with a click handler as a button — use `<button>` ; it gets keyboard support and role for free
- `data-*` attributes are not exposed to assistive technology — don't use them to convey meaningful information; use `aria-*` or semantic HTML instead
- If you must use `<div role="button">`, also add `tabindex="0"` and keyboard event handlers — a significant amount of work that `<button>` handles automatically

COMMON MISTAKES

- "Div soup" — nesting `<div>` s six layers deep when semantic elements would do the job
- Using `<div>` for interactive elements (clickable cards, fake buttons) without role or keyboard support
- Nesting a block element (`<div>` , `<p>`) inside an inline element (`<span>` , `<a>`)
- Using `data-*` to store info that should be in `aria-*` — screen readers don't read `data-*`
- Relying on CSS `display: block` on a `<span>` instead of using a `<div>` — confusing and fragile

EXAMPLE

```
<!-- ① div: block-level layout wrapper — no semantic meaning -->
<div class="card">
  <div class="card__header">
    <h2><!-- card title --></h2>
  </div>
  <div class="card__body">
    <!-- card body content -->
  </div>
</div>

<!-- ② span: inline wrapper — targets a portion of text -->
<p>The price is <span class="price">$19.99</span> per month.</p>

<!-- ③ data-* attributes: store custom data on any element -->
<button
  data-action="add-to-cart"
  data-product-id="sku-447"
  data-price="19.99">
  Add to Cart
</button>

<!-- JS access via dataset API:
  btn.dataset.action    → "add-to-cart"
  btn.dataset.productId → "sku-447"   (hyphen becomes camelCase)
  btn.dataset.price     → "19.99"
-->

<!-- ④ data-* for CSS state targeting (no JS required) -->
<div data-theme="dark">
  <!-- content inheriting dark theme styles -->
</div>

<!-- CSS: [data-theme="dark"] { background: #111; color: #eee; } -->

<!-- ⑤ Common div patterns -->

<!-- Layout wrapper -->
<div class="grid">
  <div class="grid__item"><!-- item content --></div>
  <div class="grid__item"><!-- item content --></div>
</div>

<!-- JS mount point -->
<div id="map-container"></div>

<!-- ⑥ INVALID — block element nested inside inline element -->
<!-- <span><div>Wrong!</div></span> ← browser repairs it badly -->

<!-- ⑦ WRONG — div as a fake button -->
<!-- <div onclick="save()">Save</div> ← no keyboard, no role -->
<!-- CORRECT: -->
<button type="button" onclick="save()">Save</button>
```

TIPS & BEST PRACTICES

- Reach for a semantic element first — only use `<div>` / `<span>` when nothing else fits
- Use `data-*` for JS configuration and CSS state hooks — they're stable across refactors
- Give layout `<div>` s class names that describe their role (`card` , `grid` , `modal-overlay`)
- Use `id` sparingly on `<div>` s — prefer `class` for styling, `data-*` for JS hooks

QUICK SUMMARY

- `<div>` = block generic container; `<span>` = inline generic container
- Neither carries semantic meaning — always prefer a semantic element
- `data-*` stores custom data; accessed in JS via `element.dataset.camelCase`
- Never put block elements inside inline elements — it's invalid HTML

DESCRIPTION

class and id are the primary hooks that connect HTML elements to CSS styling and JavaScript behavior. A class can be used on any number of elements and multiple classes can live on one element — making it the right tool for reusable styles. An id must be unique per page — making it the right tool for one-of-a-kind targets like a skip link anchor, a JS hook for a specific widget, or a scroll destination. Getting these right is the foundation of maintainable CSS.

NOTES

- Multiple classes are space-separated on the element: `class="card card--featured active"`
- id must be unique on the page — duplicate IDs break CSS, JS (`getElementById`), and anchor links
- id has higher specificity than class in CSS — an #id rule overrides a .class rule
- Names are case-sensitive in CSS selectors: `.Card` ≠ `.card`
- Names cannot start with a digit: `.2col` is invalid; use `.col-2`

CLASS VS ID

	CLASS	ID
Reusable?	Yes — many elements	No — unique per page
Multiple per element?	Yes — space-separated	No — one id only
CSS selector	.class-name	#id-name
JS selector	querySelectorAll('x')	getElementById('x')
CSS specificity	0-1-0 (low)	1-0-0 (high)
Anchor target?	No	Yes — href="#id"
Best for	Styles, repeated patterns	Unique widgets, anchors, JS

NAMING CONVENTIONS

CONVENTION	EXAMPLE	USE
kebab-case	nav-link	Standard for HTML classes
BEM Block	card	Component root
BEM Element	card__title	Part of a component
BEM Modifier	card--featured	State or variant
JS hook	js-modal-trigger	For JS only, never styled

ACCESSIBILITY

- id is used to associate labels with form controls (`<label for="id">`) and to link ARIA descriptions (`aria-describedby="id"`)
- Duplicate IDs silently break `aria-labelledby` and `aria-describedby` associations — always validate for uniqueness
- The `class="sr-only"` pattern (visually hidden but accessible) is a critical ally tool — keep a utility class for it in your CSS
- Never use a class name to convey meaning to users — that information must also be in the HTML semantics or ARIA

COMMON MISTAKES

- Reusing the same id on multiple elements — only the first is found by `getElementById` and CSS
- Using id for styling everything — high specificity makes it painful to override later
- Starting class names with a number — `.2up-grid` is invalid; use `.grid-2up`
- Using the same class for both JS behavior and CSS styling — a refactor of one breaks the other
- Forgetting that class names are case-sensitive — `.NavLink` and `.navlink` are different selectors

EXAMPLE

```
<!-- 1) class: reusable — multiple elements,
      multiple classes per element -->
<div class="card">
  <h2 class="card__title"><!-- title --></h2>
  <p class="card__body"><!-- body --></p>
</div>

<!-- BEM modifier: adds a variant without duplicating base styles -->
<div class="card card--featured">
  <h2 class="card__title"><!-- title --></h2>
  <p class="card__body"><!-- body --></p>
</div>

<!-- CSS: .card { ... } .card--featured { border-color: gold; } -->

<!-- 2) id: unique anchor target + JS hook -->
<section id="pricing">
  <!-- section content -->
</section>

<!-- Scroll link from anywhere on the page: -->
<a href="#pricing">See Pricing</a>

<!-- 3) id linking a label to its input (critical for ally) -->
<label for="email-input">Email address</label>
<input type="email" id="email-input" name="email" />

<!-- 4) aria-describedby links an id to a hint or error message -->
<input type="password" id="pwd" aria-describedby="pwd-hint" />
<p id="pwd-hint">Must be at least 8 characters.</p>

<!-- 5) js- prefix: JS-only hook, never styled in CSS -->
<button class="btn js-open-modal" data-target="contact-modal">
  Contact Us
</button>

<!-- 6) sr-only: visually hidden but readable by screen readers -->
<span class="sr-only">Current page:</span>
<!-- CSS: .sr-only { position:absolute; width:1px; height:1px;
      overflow:hidden; clip:rect(0,0,0,0);
      white-space:nowrap; } -->

<!-- 7) Multiple classes for active state -->
<li class="nav-item nav-item--active">
  <a href="/dashboard" class="nav-link">Dashboard</a>
</li>
```

TIPS & BEST PRACTICES

- Use class for styles, id for unique targets, data-* for JS configuration — keep them separate
- Prefix JS-only classes with js- so they're never accidentally removed during CSS refactors
- Keep a .sr-only utility class in every project — you'll need it for accessible icon buttons and skip links
- Use BEM or another consistent naming convention — it prevents specificity conflicts as projects grow

QUICK SUMMARY

- class = reusable, many elements; id = unique, one element per page
- CSS: .class selector; #id selector — id has higher specificity
- id enables anchor links, label associations, and ARIA references
- Separate styling classes, JS hooks (js-), and ARIA IDs — don't let them cross-contaminate

DESCRIPTION

HTML provides a rich set of text-level elements that give meaning to written content. The key distinction is *semantic* vs *presentational*: `<strong>` means "this is important" (screen readers stress it); `<b>` means "bold visually" (no semantic weight). Always choose the element that describes *what the text means*, not how you want it to look — CSS handles appearance. Heading levels (`<h1>`–`<h6>`) define the document outline used by search engines, assistive technology, and browser reading modes.

NOTES

- One `<h1>` per page — it is the document title; all other headings descend from it
- Never skip heading levels for visual sizing — use CSS for that; skipping `<h2>` to `<h4>` breaks the outline
- `<strong>` and `<em>` are semantic; `<b>` and `<i>` are presentational — prefer the semantic versions
- `<pre>` preserves whitespace and line breaks exactly as written — used to wrap `<code>` blocks
- `<abbr title>` exposes the full form on hover and to screen readers — use it on first occurrence

INLINE TEXT ELEMENTS

ELEMENT	MEANING	DEFAULT RENDER
<code></code>	Strong importance	Bold
<code></code>	Stress emphasis	Italic
<code></code>	Stylistically bold, no meaning	Bold
<code><i></code>	Offset text (term, thought)	Italic
<code><mark></code>	Highlighted / search result	Yellow highlight
<code></code>	Deleted text	Strikethrough
<code><ins></code>	Inserted text	Underline
<code><sub></code>	Subscript	Below baseline, smaller
<code><sup></code>	Superscript	Above baseline, smaller
<code><code></code>	Inline code fragment	Monospace
<code><kbd></code>	Keyboard input	Monospace
<code><abbr></code>	Abbreviation with expansion	Dotted underline

ACCESSIBILITY

- Screen readers announce `<strong>` with increased stress and `<em>` with emphasis — `<b>` and `<i>` are ignored
- Heading levels define the document outline — screen reader users navigate pages by jumping between headings; a broken outline is a major barrier
- `<abbr title="...">` exposes the expansion to screen readers and on hover — use on first occurrence of every abbreviation
- Never use headings purely for visual size — a user relying on heading navigation will be confused by out-of-order levels

COMMON MISTAKES

- Skipping heading levels for visual effect — jumping from `<h2>` to `<h4>` breaks the document outline
- Using multiple `<h1>` elements — one per page is the standard; it signals the single main topic
- Using `<b>` or `<i>` when `<strong>` or `<em>` is semantically correct
- Using `<code>` without `<pre>` for multi-line blocks — whitespace collapses without `<pre>`
- Using headings to style decorative text — use CSS classes instead

TIPS & BEST PRACTICES

- Build your heading outline first, without any CSS — it should read like a table of contents
- Use `<strong>` sparingly — if everything is important, nothing is
- Use `<abbr>` on first occurrence only — repeating it on every instance is noisy
- Pair `<del>` + `<ins>` with `aria-label` or screen-reader text since strikethrough is visual-only

QUICK SUMMARY

- One `<h1>` per page; headings `<h2>` – `<h6>` must descend logically — never skip levels
- `<strong>` / `<em>` = semantic weight; `<b>` / `<i>` = visual only
- `<pre><code>` for multi-line code; `<code>` alone for inline fragments
- Choose elements for meaning first — CSS will handle the look

EXAMPLE

```
<!-- ① Heading hierarchy – one h1, then descend logically -->
<h1>The Guide to HTML Typography</h1>
  <h2>Text Elements</h2>
    <h3>Semantic vs Presentational</h3>
<!-- Never skip levels (h2 → h4) just for visual sizing – use CSS -->

<!-- ② Paragraph -->
<p><!-- default block of body text --></p>

<!-- ③ Semantic emphasis – read by screen readers with stress -->
<p>
  This step is <strong>required</strong> before continuing.
  Read <em>all</em> instructions carefully.
</p>

<!-- ④ Presentational – visual only, no screen reader stress -->
<p>
  The term <i>terroir</i> refers to environmental factors.
  Significant results shown in <b>bold</b>.
</p>

<!-- ⑤ mark: highlighted match (e.g. search result) -->
<p>Results for <mark>semantic HTML</mark> – 42 articles found.</p>

<!-- ⑥ del + ins: tracked changes, price updates -->
<p>Was <del>$49/mo</del> now <ins>$29/mo</ins></p>

<!-- ⑦ sub + sup -->
<p>Water is H<sub>2</sub>O. E = mc<sup>2</sup></p>
<p>See footnote<sup><a href="#fn1">1</a></sup></p>

<!-- ⑧ code: inline code fragment -->
<p>Use <code>querySelector()</code> to find elements.</p>

<!-- ⑨ pre + code: multi-line block – whitespace preserved exactly -->
<pre><code>function greet(name) {
  return `Hello, ${name}!`;
}
greet('World'); // "Hello, World!"</code></pre>

<!-- ⑩ kbd: keyboard instruction -->
<p>Save with <kbd>Ctrl</kbd> + <kbd>S</kbd></p>

<!-- ⑪ abbr: expand abbreviation on first use -->
<p>
  <abbr title="HyperText Markup Language">HTML</abbr> is the
  language of the web.
</p>
```

DESCRIPTION

HTML has three list types: `<ul>` (unordered — bullets, order doesn't matter), `<ol>` (ordered — numbered, sequence matters), and `<dl>` (description list — term/definition pairs). Lists are one of the most semantically important structures in HTML: navigation menus, breadcrumbs, feature lists, and step-by-step instructions should all use list markup, not bare `<div>`s. Screen readers announce the list type and item count, which helps users understand content structure before reading it.

NOTES

- ▶ The only valid direct children of `<ul>` and `<ol>` are `<li>` elements
- ▶ `<ol>` attributes: `type` (l, A, a, l, i), `start` (starting number), `reversed` (count down)
- ▶ `<li>` inside an `<ol>` can have a `value` attribute to override its number
- ▶ `<dl>` allows multiple `<dt>` per `<dd>` and vice versa — useful for glossaries
- ▶ CSS `list-style: none` on a `<ul>` removes the bullet and the list semantics in Safari — add `role="list"` to restore them

WHICH LIST TYPE?

USE	ELEMENT	EXAMPLE
Order doesn't matter	<code></code>	Features, ingredients, nav links
Order matters	<code></code>	Steps, rankings, instructions
Term + definition pairs	<code><dl></code>	Glossary, metadata, FAQ

 ATTRIBUTES

ATTRIBUTE	VALUES	EFFECT
<code>type</code>	l A a l i	Marker style: numbers, letters, Roman
<code>start</code>	any integer	Starting counter value
<code>reversed</code>	boolean	Count down instead of up
<code>value (on li)</code>	integer	Override this item's counter

3

ACCESSIBILITY

- ▶ Screen readers announce "list, N items" when entering a list — this orientation is lost with `<div>`s
- ▶ Safari removes list semantics when `list-style: none` is applied — add `role="list"` on the `<ul>` to restore the announcement
- ▶ Navigation menus should be `<nav><ul>` — screen readers announce "navigation" then "list" giving users immediate context
- ▶ `<dl>` pairs are announced as definition term / definition — ideal for key-value metadata (author, date, category)

QUICK SUMMARY

- ▶ `<ul>` = unordered; `<ol>` = ordered; `<dl>` = term/definition pairs
- ▶ Only `<li>` is a valid direct child of `<ul>` and `<ol>`
- ▶ Nav menus = `<nav><ul><li><a>` — always use list markup
- ▶ Add `role="list"` when `list-style: none` is applied — preserves Safari screen reader semantics

⚠COMMON MISTAKES

- ▶ Putting elements other than `<li>` as direct children of `<ul>` / `<ol>` — invalid HTML
- ▶ Using `<ol>` for items where order doesn't matter — misleads screen readers who announce "list of steps"
- ▶ Building navigation with `<div>`s instead of `<nav><ul><li>` — loses landmark and list semantics
- ▶ Forgetting `role="list"` when using `list-style: none` in CSS — Safari drops list semantics silently
- ▶ Using a `<dl>` for generic two-column layouts — it's specifically for term/definition relationships

✓TIPS & BEST PRACTICES

- ▶ Always wrap navigation links in `<nav><ul><li>` — even if CSS makes them look horizontal
- ▶ Use `<dl>` for any key-value metadata — author, date, tags, file size — it's more semantic than a table
- ▶ Add `role="list"` when applying `list-style: none` to a `<ul>` — takes two seconds, preserves ally
- ▶ Nest lists inside the `<li>` (not after it) — the sub-list is part of that item

EXAMPLE

```
<!-- ① ul: unordered — order doesn't matter -->
<ul>
  <li><!-- item --></li>
  <li><!-- item --></li>
  <li><!-- item --></li>
</ul>

<!-- ② ol: ordered — sequence matters -->
<ol>
  <li>Install Node.js</li>
  <li>Run <code>npm install</code></li>
  <li>Start the server with <code>npm run dev</code></li>
</ol>

<!-- ③ ol attributes: type, start, reversed -->
<ol type="A" start="3" reversed>
  <li><!-- renders as "E" counting down --></li>
  <li><!-- renders as "D" --></li>
</ol>

<!-- ④ Nested list — child ul lives INSIDE the parent li -->
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>

<!-- ⑤ dl: description / definition list -->
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language — the structure of web pages.</dd>

  <!-- multiple dt elements can share one dd -->
  <dt>JS</dt>
  <dt>JavaScript</dt>
  <dd>The programming language of the web.</dd>
</dl>

<!-- ⑥ Navigation — ul is the standard pattern for nav links -->
<nav aria-label="Primary">
  <!-- role="list" restores Safari semantics lost by list-style:none -->
  <ul role="list">
    <li><a href="/">Home</a></li>
    <li><a href="/about">About</a></li>
    <li><a href="/contact">Contact</a></li>
  </ul>
</nav>

<!-- ⑦ dl as a metadata block -->
<dl>
  <dt>Author</dt> <dd>Jane Smith</dd>
  <dt>Published</dt><dd><time datetime="2025-06-01">June 1, 2025</time></dd>
  <dt>Category</dt> <dd>Web Development</dd>
</dl>
```

< Typography

[Table of Contents]

Tables >

DESCRIPTION

HTML tables are for *tabular data* — information that has a meaningful relationship between rows and columns, like a spreadsheet or schedule. They are **not** for page layout (that's CSS Grid and Flexbox). A well-structured table uses `<thead>`, `<tbody>`, and `<caption>` to communicate its structure to both humans and assistive technology. The `scope` attribute on `<th>` tells screen readers which cells a header applies to — essential for accessible tables.

NOTES

- `<caption>` must be the first child of `<table>` — it is the table's accessible name
- `<th>` is a header cell; `<td>` is a data cell — never use `<td>` for header content
- `colspan` spans columns; `rowspan` spans rows — remove the cells that the spanning cell covers
- `<tfoot>` can appear before `<tbody>` in source — the browser always renders it at the bottom
- `<colgroup>` + `<col>` let you apply styles to entire columns without touching each cell

TABLE ELEMENT REFERENCE

ELEMENT	PURPOSE
<code><table></code>	Root table element
<code><caption></code>	Table title — must be first child, becomes accessible name
<code><colgroup></code>	Groups columns for styling
<code><col></code>	Single column inside colgroup; use span to cover multiple
<code><thead></code>	Header row group — repeated on print page breaks
<code><tbody></code>	Body row group — main data rows
<code><tfoot></code>	Footer row group — totals, summaries
<code><tr></code>	Table row
<code><th></code>	Header cell — bold, centered by default; add scope
<code><td></code>	Data cell

SCOPE ATTRIBUTE VALUES

VALUE	HEADER APPLIES TO...
<code>col</code>	All cells in the same column below
<code>row</code>	All cells in the same row to the right
<code>colgroup</code>	All cells in the colgroup
<code>rowgroup</code>	All cells in the row group (thead/tbody/tfoot)

ACCESSIBILITY

- Always include `<caption>` — it is the accessible name of the table and the first thing screen readers announce
- Use `scope="col"` on column headers and `scope="row"` on row headers — allows screen readers to associate cells with their headers
- For complex tables with irregular headers, use `id` on each `<th>` and `headers="id1 id2"` on the corresponding `<td>`
- Never use a table for layout — screen readers announce "table, N columns" which confuses users when there's no data relationship

COMMON MISTAKES

- Using tables for page layout — they make code unmaintainable and are a major accessibility failure
- Omitting `<caption>` — screen readers announce "table" with no label, leaving users without context
- Forgetting to remove the cells covered by `colspan` / `rowspan` — causes extra phantom cells
- Using `<td>` for header cells — loses bold styling and, more importantly, the screen reader announcement
- Missing `scope` on `<th>` elements — screen readers can't associate headers with data cells

1 FULL TABLE STRUCTURE

```
<table>
  <!-- caption: accessible name —
        must be first child -->
  <caption>Q1 2025 Sales by Region</caption>

  <!-- colgroup: style entire columns via CSS -->
  <colgroup>
    <col />                <!-- Region -->
    <col class="highlight" /> <!-- Jan -->
    <col span="2" />         <!-- Feb + Mar -->
    <col class="total-col" /> <!-- Total -->
  </colgroup>

  <thead>
    <tr>
      <!-- scope="col": header applies
            to cells below -->
      <th scope="col">Region</th>
      <th scope="col">Jan</th>
      <th scope="col">Feb</th>
      <th scope="col">Mar</th>
      <th scope="col">Total</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <!-- scope="row": header applies
            to cells to its right -->
      <th scope="row">North</th>
      <td>$42,000</td>
      <td>$38,500</td>
      <td>$51,200</td>
      <td>$131,700</td>
    </tr>
    <tr>
      <th scope="row">South</th>
      <td><!-- data cells --></td>
      <td></td><td></td><td></td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <th scope="row">Total</th>
      <td><!-- column totals --></td>
      <td></td><td></td>
      <td><strong><!-- grand total --></strong></td>
    </tr>
  </tfoot>
</table>
```

2 COLSPAN + ROWSPAN

```
<table>
  <caption>Conference Schedule</caption>
  <thead>
    <tr>
      <th scope="col">Time</th>
      <th scope="col">Track A</th>
      <th scope="col">Track B</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">9:00 AM</th>
      <!-- colspan="2": spans both track columns -->
      <td colspan="2">Opening Keynote</td>
    </tr>
    <tr>
      <th scope="row">10:00 AM</th>
      <td>HTML Deep Dive</td>
      <!-- rowspan="2": covers the 11 AM row too -->
      <td rowspan="2">CSS Workshop</td>
    </tr>
    <tr>
      <th scope="row">11:00 AM</th>
      <td>JS Performance</td>
      <!-- NO td here — rowspan above covers it -->
    </tr>
  </tbody>
</table>
```

▶ RENDERED TABLE 1

Q1 2025 Sales by Region				
REGION	JAN	FEB	MAR	TOTAL
North	\$42,000	\$38,500	\$51,200	\$131,700
South	—	—	—	—
Total	—	—	—	—

▶ RENDERED TABLE 2

Conference Schedule		
TIME	TRACK A	TRACK B
9:00 AM	Opening Keynote	
10:00 AM	HTML Deep Dive	CSS Workshop
11:00 AM	JS Performance	

✓ TIPS & BEST PRACTICES

- Always include `<thead>`, `<tbody>`, and `<caption>` — minimum structure for an accessible table
- Add `scope="col"` to all column headers and `scope="row"` to all row headers — takes seconds, big ally win
- Use `<colgroup>` to style columns — far cleaner than adding classes to every `<td>`
- Test with a screen reader — navigate cell by cell to hear if headers are correctly associated

QUICK SUMMARY

- Tables = tabular data only, never layout
- Always include `<caption>` (table name), `<thead>`, `<tbody>`
- Use `scope="col"/row"` on every `<th>` for accessible header association
- `colspan` / `rowspan` merge cells — remove the cells they replace

DESCRIPTION

This sheet covers elements that don't belong to the major structural, text, or media families but are essential tools in a working HTML toolkit. `<br>` and `<hr>` are small but often misused. `<details>` and `<dialog>` are native browser interactive patterns that work without any JavaScript. `<template>` holds inert markup for JavaScript to clone at runtime. `<noscript>` provides graceful degradation when JavaScript is unavailable.

NOTES

- › `<br>` is for meaningful line breaks (poetry, addresses) — not for adding vertical space between elements (use CSS margin)
- › `<hr>` is a *thematic break* — block-level despite often appearing inline in context
- › `<details>` works without JS — the `open` boolean attribute controls its initial state
- › `<dialog>` has two modes: modal (`showModal()` — traps focus, adds backdrop) and non-modal (`show()`)
- › Content inside `<template>` is completely inert — scripts don't run, images don't load until it's cloned into the DOM

① BR ② HR ③ WBR

```
<!-- ① br: meaningful line break
      (address, poetry) -->
<address>
  123 Main Street<br />
  Springfield, IL 62701<br />
  USA
</address>

<!-- ② hr: thematic section break
      (block-level) -->
<section>
  <h2>Chapter 1</h2>
  <!-- chapter content -->
</section>
<hr />
<section>
  <h2>Chapter 2</h2>
  <!-- chapter content -->
</section>

<!-- ③ wbr: optional word-break hint
      for long strings -->
<p>
  https://www.example.com<wbr />
  /very<wbr />/long<wbr />/path
</p>
```

④ DETAILS + SUMMARY ⑤ DIALOG

```
<!-- ④ details + summary: native accordion — no JS -->
<details>
  <summary>What is your refund policy?</summary>
  <p><!-- answer content --></p>
</details>

<!-- open attribute: expanded by default -->
<details open>
  <summary>Expanded on load</summary>
  <p><!-- content visible immediately --></p>
</details>

<!-- ⑤ dialog: showModal() traps focus + adds backdrop -->
<dialog id="confirm-dialog" aria-labelledby="dialog-title">
  <h2 id="dialog-title">Confirm Delete</h2>
  <p>Are you sure? This cannot be undone.</p>
  <div>
    <button id="cancel-btn">Cancel</button>
    <button id="confirm-btn">Delete</button>
  </div>
</dialog>
<button id="open-btn">Delete Item</button>

<script defer>
  const dialog = document.getElementById('confirm-dialog');
  document.getElementById('open-btn')
    .addEventListener('click', () => dialog.showModal());
  document.getElementById('cancel-btn')
    .addEventListener('click', () => dialog.close('cancelled'));
  document.getElementById('confirm-btn')
    .addEventListener('click', () => dialog.close('confirmed'));
</script>
```

⑥ TEMPLATE ⑦ NOSCRIPT

```
<!-- ⑥ template: inert —
not rendered until cloned by JS -->
<template id="card-template">
  <div class="card">
    <h3 class="card__title"></h3>
    <p class="card__body"></p>
  </div>
</template>

<script>
  const clone = document
    .getElementById('card-template')
    .content.cloneNode(true);
  clone.querySelector('.card__title')
    .textContent = 'Hello';
  document.body.appendChild(clone);
</script>

<!-- ⑦ noscript: shown only
      when JS is disabled -->
<noscript>
  <p>
    This app requires JavaScript.
    Please enable it to continue.
  </p>
</noscript>
```

ELEMENT QUICK REFERENCE

ELEMENT	TYPE	PURPOSE
<code>
</code>	Void / inline	Meaningful line break within a paragraph
<code><hr></code>	Void / block	Thematic break between content sections
<code><wbr></code>	Void / inline	Optional word-break hint for long strings
<code><details></code>	Block	Native disclosure / accordion — no JS needed
<code><summary></code>	Block	Visible label/trigger for <code><details></code>
<code><dialog></code>	Block	Native modal or non-modal dialog
<code><template></code>	Inert	Cloneable HTML fragment; not rendered until cloned
<code><noscript></code>	Block	Fallback content shown when JS is disabled

<DIALOG> JS API

METHOD / EVENT	DESCRIPTION
<code>showModal()</code>	Open as modal — traps focus, adds ::backdrop
<code>show()</code>	Open as non-modal — no focus trap
<code>close(val)</code>	Close; optional return value stored in returnValue
<code>open attribute</code>	Boolean — present when dialog is open
<code>close event</code>	Fires when dialog is closed
<code>cancel event</code>	Fires on Escape key press

♿ ACCESSIBILITY

- › `<details>` / `<summary>` are natively keyboard accessible — Tab to focus, Enter/Space to toggle; no ARIA needed
- › `<dialog>` opened with `showModal()` automatically traps focus and sets `aria-modal` — prefer it over a custom modal div
- › Give every `<dialog>` an accessible name via `aria-label` or `aria-labelledby` pointing to a heading inside it
- › `<hr>` has an implicit role of "separator" — announced by screen readers as a section boundary

⚠ COMMON MISTAKES

- › Using `<br>` tags for vertical spacing between elements — use CSS `margin` instead
- › Building a custom modal `<div>` with JS focus-trapping instead of using native `<dialog>`
- › Opening `<dialog>` with `show()` when `showModal()` is needed — non-modal dialogs don't trap focus
- › Forgetting `aria-labelledby` on a `<dialog>` — screen readers announce "dialog" with no context
- › Expecting scripts or images inside `<template>` to load — they're completely inert until cloned

✓ TIPS & BEST PRACTICES

- › Use `<details>` / `<summary>` for FAQs and simple accordions — zero JavaScript required
- › Always use `showModal()` over `show()` for confirmation dialogs — focus trapping and backdrop are critical UX
- › Style `<dialog>`'s `::backdrop` pseudo-element to control the overlay appearance
- › Use `<template>` for repeated UI patterns (cards, list items) rendered by JS — cleaner than building HTML strings

QUICK SUMMARY

- › `<br>` = meaningful line break; `<hr>` = thematic section break
- › `<details>` / `<summary>` = native accordion, no JS; `open` attribute = expanded by default
- › `<dialog>` + `showModal()` = native modal with focus trap and backdrop
- › `<template>` holds inert markup; clone with `content.cloneNode(true)`

DESCRIPTION

HTML embeds images, video, audio, and other content through a family of dedicated elements. `<img>` is the most used — and the most often done wrong. Responsive images (`srcset`, `sizes`, `<picture>`) let the browser pick the best asset for the device. `<video>` and `<audio>` provide native playback controls. `<iframe>` embeds external content. `<canvas>` is a pixel-drawing surface for JavaScript. Each has specific performance, security, and accessibility requirements.

NOTES

- `alt` on `<img>` is mandatory — omitting it is an accessibility violation; use `alt=""` for decorative images
- Always set `width` and `height` on `<img>` — the browser reserves space before the image loads, preventing layout shift
- `autoplay` on `<video>` requires `muted` — browsers block autoplay with audio by default
- `<iframe>` — always add `sandbox` for untrusted content; it restricts scripts, forms, and top-level navigation
- Prefer modern formats: WebP or AVIF over JPEG/PNG for photos; SVG for logos and icons

① IMG ② DECORATIVE ③ SRCSET

```
<!-- ① Basic image —
      alt, width, height always required -->


<!-- ② Decorative image — alt=""
      tells screen readers to skip -->


<!-- ③ srcset: resolution switching —
      same image, different sizes -->

```

④ PICTURE ⑤ VIDEO

```
<!-- ④ picture: art direction —
      different crops per breakpoint -->
<picture>
  <!-- mobile crop, modern format first -->
  <source media="(max-width: 480px)"
          srcset="hero-mobile.avif" type="image/avif" />
  <source media="(max-width: 480px)"
          srcset="hero-mobile.webp" type="image/webp" />
  <!-- desktop crop -->
  <source srcset="hero-desktop.avif" type="image/avif" />
  <source srcset="hero-desktop.webp" type="image/webp" />
  <!-- img is the required fallback — always last -->
  
</picture>

<!-- ⑤ video with controls, poster, captions -->
<video controls width="854" height="480"
        poster="poster.jpg" preload="metadata">
  <source src="intro.mp4" type="video/mp4" />
  <source src="intro.webm" type="video/webm" />
  <!-- captions required for accessibility -->
  <track kind="captions" src="intro-en.vtt"
        srclang="en" label="English" default />
  <p>Your browser doesn't support video.
    <a href="intro.mp4">Download</a></p>
</video>
```

⑥ AUDIO ⑦ IFRAME ⑧ CANVAS

```
<!-- ⑥ audio -->
<audio controls preload="none">
  <source src="podcast.mp3" type="audio/mpeg" />
  <source src="podcast.ogg" type="audio/ogg" />
</audio>

<!-- ⑦ iframe — title required;
      sandbox for untrusted content -->
<iframe
  src="https://maps.example.com/embed?q=NYC"
  title="Map of New York City"
  width="600" height="450"
  loading="lazy"
  sandbox="allow-scripts allow-same-origin">
</iframe>

<!-- ⑧ canvas — set size via attributes,
      not CSS -->
<canvas id="chart" width="600" height="300">
  <!-- fallback for no-canvas environments -->
  <p>Bar chart: monthly sales. Text data below.</p>
</canvas>
```

IMAGE FORMAT GUIDE

FORMAT	BEST FOR	NOTES
AVIF	Photos, complex images	Best compression; growing support
WebP	Photos, illustrations	Great compression; excellent support
SVG	Icons, logos, diagrams	Scales infinitely; styleable with CSS
PNG	Transparency needed	Lossless; large files
JPEG	Photos (legacy)	Lossy; no transparency; widely supported
GIF	Simple animation (legacy)	Prefer <code><video></code> or CSS for animation

KEY ATTRIBUTES BY ELEMENT

ELEMENT	KEY ATTRIBUTES
<code></code>	src, alt*, width, height, loading, decoding, srcset, sizes
<code><video></code>	src, controls, autoplay, muted, loop, poster, preload, width, height
<code><audio></code>	src, controls, autoplay, muted, loop, preload
<code><source></code>	src, type, media, srcset (inside picture)
<code><iframe></code>	src, title*, width, height, sandbox, allow, loading
<code><canvas></code>	width, height (set via attr not CSS); fallback content inside

③ ACCESSIBILITY

- Informative images: `alt` describes what the image conveys. Decorative images: `alt=""` (empty string, not missing)
- `<video>` requires captions via `<track kind="captions">` for deaf users and transcripts for deafblind users
- `<iframe>` must have a descriptive `title` attribute — screen readers announce it as the frame's label
- `<canvas>` has no built-in accessibility — put fallback content inside it and consider an ARIA description or offscreen text alternative

⚠ COMMON MISTAKES

- Omitting `alt` on `<img>` — not the same as `alt=""`; missing entirely is an a11y violation
- Not setting `width` / `height` on images — causes cumulative layout shift (CLS) as images load
- Using `autoplay` without `muted` — browsers block it and the video silently fails to play
- Setting `<canvas>` size via CSS instead of `width` / `height` attributes — distorts the drawing surface
- Using `<iframe>` without `title` — screen reader users hear "frame" with no context

✓ TIPS & BEST PRACTICES

- Use `loading="lazy"` on all below-the-fold images — free performance win, widely supported
- Use `<picture>` with AVIF + WebP + JPEG sources for maximum compression with fallback support
- Always include a `poster` on `<video>` — shows a meaningful thumbnail before playback
- Add `sandbox` to all third-party `<iframe>`s — meaningful security boundary with minimal cost

QUICK SUMMARY

- `<img>` always needs `alt`, `width`, `height` — use `alt=""` for decorative images
- `srcset` + `sizes` = resolution switching; `<picture>` = art direction
- `autoplay` requires `muted`; always provide `<source>` fallbacks for video/audio
- `<iframe>` needs `title` + `sandbox`; `<canvas>` size set via attributes not CSS

DESCRIPTION

The anchor element `<a>` is how HTML connects documents, sections, email addresses, phone numbers, and downloadable files. The `href` attribute determines the destination — it can be an absolute URL, a relative path, a same-page fragment, or a special scheme like `mailto:` or `tel:`. Link text must describe the destination, not just say "click here." When opening links in a new tab, always pair `target="_blank"` with `rel="noopener noreferrer"` to close a security vulnerability.

NOTES

- `<a>` without `href` is a placeholder link — it has no role and is not focusable
- `target="_blank"` without `rel="noopener"` gives the opened page access to your page via `window.opener` — a security hole
- The `download` attribute works only for same-origin URLs or data URLs — cross-origin links are navigated instead
- Fragment links (`href="#id"`) move focus to the target element — it must exist and the target should be focusable or have `tabindex="-1"`
- Avoid `href="javascript:void(0)"` — use a `<button>` for actions that don't navigate

HREF VALUE TYPES

TYPE	EXAMPLE	NAVIGATES TO
Absolute URL	<code>https://example.com/page</code>	Full URL, any origin
Root-relative	<code>/about</code>	Same origin, from root
Relative path	<code>../contact</code>	Relative to current file
Fragment	<code>#section-id</code>	Same-page anchor by id
<code>mailto:</code>	<code>mailto:hi@example.com</code>	Opens email client
<code>tel:</code>	<code>tel:+15555550100</code>	Opens phone dialer
<code>sms:</code>	<code>sms:+15555550100</code>	Opens SMS app

REL ATTRIBUTE VALUES

VALUE	PURPOSE
<code>noopener</code>	Blocks opener access — always use with <code>target="_blank"</code>
<code>noreferrer</code>	Hides referrer header + implies <code>noopener</code>
<code>nofollow</code>	Tells search engines not to follow this link
<code>external</code>	Indicates the link goes to an external site
<code>prev / next</code>	Pagination relationships for SEO
<code>author</code>	Links to the author of the document

ACCESSIBILITY

- Link text must describe the destination — "click here" and "read more" are meaningless to screen readers navigating by link list
- If link text is an icon or image, add `aria-label` or descriptive `alt` text — e.g. `<a href="#"><img alt="GitHub profile"></a>`
- Warn users when a link opens in a new tab — add visible text or `aria-label="... (opens in new tab)"`
- Fragment links must move keyboard focus to the target — add `tabindex="-1"` to non-interactive targets like `<h2>`

COMMON MISTAKES

- Using `target="_blank"` without `rel="noopener noreferrer"` — opens a security vulnerability via `window.opener`
- Non-descriptive link text — "click here", "read more", "here" tell screen reader users nothing about the destination
- Using `<a>` with `href="javascript:void(0)"` for buttons — use `<button>` instead
- Expecting `download` to work on cross-origin URLs — browsers navigate instead of downloading
- Using `tel:` with spaces or dashes in the number — the href value should be digits only: `tel:+15555550100`

TIPS & BEST PRACTICES

- Every external link should have `rel="noopener noreferrer"` — make it a default in your templates
- Write link text that makes sense out of context — imagine a screen reader reading only the links on the page
- Use root-relative paths (`/about`) for internal links — they work regardless of the current page's depth
- Add a visible "(opens in new tab)" hint or icon alongside `target="_blank"` — sighted users appreciate the warning too

EXAMPLE

```
<!-- ① Absolute URL -->
<a href="https://developer.mozilla.org">MDN Web Docs</a>

<!-- ② Root-relative (same origin, from site root) -->
<a href="/about">About Us</a>

<!-- ③ Relative path (relative to current file) -->
<a href="../pricing">Pricing</a>

<!-- ④ Fragment - scrolls to element with matching id -->
<a href="#contact">Jump to Contact</a>
<!-- ...target element on the same page... -->
<section id="contact">
  <h2>Contact</h2>
  <!-- contact content -->
</section>

<!-- ⑤ New tab - always add rel for security -->
<a href="https://example.com"
  target="_blank"
  rel="noopener noreferrer">
  Visit Example.com (opens in new tab)
</a>

<!-- ⑥ mailto: with subject and body prefilled -->
<a href="mailto:hello@example.com
  ?subject=Project%20Inquiry
  &body=Hi%2C%20I'd%20like%20to%20discuss...">
  Email us
</a>

<!-- ⑦ tel: phone link (mobile-friendly) -->
<a href="tel:+15555550100">(555) 555-0100</a>

<!-- ⑧ download: force file download, optionally rename it -->
<a href="/reports/q1-2025.pdf" download="Q1-2025-Report.pdf">
  Download Q1 Report (PDF)
</a>

<!-- ⑨ Icon-only link - aria-label provides the accessible name -->
<a href="https://github.com/username"
  aria-label="GitHub profile (opens in new tab)"
  target="_blank" rel="noopener noreferrer">
  <svg aria-hidden="true" focusable="false">
    <!-- icon paths -->
  </svg>
</a>

<!-- ⑩ WRONG - anchor used as a JS action button -->
<!-- <a href="javascript:void(0)" onclick="doThing()">Do Thing</a> -->
<!-- CORRECT - use a button for actions that don't navigate -->
<button type="button" onClick="doThing()">Do Thing</button>
```

QUICK SUMMARY

- `<a href>` navigates; `<button>` acts — don't use `<a>` for JS-only actions
- `target="_blank"` always needs `rel="noopener noreferrer"`
- Link text must describe the destination, not just say "click here"
- `mailto:`, `tel:`, `#fragment`, `download` — four href patterns beyond URLs

DESCRIPTION

Forms are the primary mechanism for collecting user input — from simple search boxes to multi-step checkouts. HTML5 provides over 20 input types, each with built-in validation, mobile keyboards, and accessibility semantics. Every input needs a `<label>` — either wrapping it or linked via `for/id`. Use `<fieldset>` + `<legend>` to group related controls. HTML's native validation (`required`, `pattern`, `min/max`) provides a solid baseline before adding JS validation.

NOTES

- method="GET" appends data to the URL — use for search; method="POST" sends in the body — use for sensitive data
- Every form control must have an associated `<label>` — `placeholder` is not a substitute
- autocomplete tokens (e.g. "email", "given-name") let password managers and browsers autofill correctly
- `<button type="button">` inside a form does NOT submit — omitting `type` defaults to `submit`
- Use `novalidate` on `<form>` only when replacing browser validation with a custom JS implementation

INPUT TYPES REFERENCE

TYPE=	PURPOSE & BEHAVIOUR
text	Single-line free text — default type
email	Email address — validates format, shows email keyboard
password	Masked text — never autocomplete="off" for passwords
number	Numeric; min, max, step attributes
tel	Phone number — shows dial keyboard, no format validation
url	URL — validates format, shows URL keyboard
search	Search field — may show clear button
date	Date picker (YYYY-MM-DD value)
time	Time picker (HH:MM value)
datetime-local	Date + time, no timezone
month	Month + year picker
range	Slider; min, max, step, value
color	Color picker; value is hex
checkbox	Toggle; checked attribute = default on
radio	One-of-many; grouped by name attribute
file	File picker; accept, multiple attributes
hidden	Not shown; submits value silently
submit	Submit button — prefer <code><button type="submit"></code>
reset	Resets form to defaults — rarely useful

ACCESSIBILITY

- Every input needs a visible `<label>` — `placeholder` disappears on typing and has low contrast; it is not a label
- Group radio buttons and checkboxes in `<fieldset>` with `<legend>` — screen readers announce the legend before each option
- Link error messages with `aria-describedby="error-id"` — screen readers read the error after the field's label when focused
- Use `aria-invalid="true"` on fields with validation errors — announced by screen readers alongside the field role

COMMON MISTAKES

- Using `placeholder` as a label — it disappears on typing and fails WCAG contrast requirements
- Omitting `type` on `<button>` inside a form — defaults to `submit`, causing accidental submissions
- Grouping radio buttons without `<fieldset>` / `<legend>` — screen readers read each option without the group question
- Using `autocomplete="off"` on password fields — prevents password managers from filling saved credentials
- Displaying validation errors without linking them to the field via `aria-describedby` — screen reader users miss the error message

EXAMPLE

```
<form action="/signup" method="post">

  <!-- ① Text + email — label linked via for/id pair -->
  <label for="name">Full name</label>
  <input type="text" id="name" name="name"
    autocomplete="name" required />

  <label for="email">Email address</label>
  <input type="email" id="email" name="email"
    autocomplete="email" required />

  <!-- ② Password with accessible hint -->
  <label for="pwd">Password</label>
  <input type="password" id="pwd" name="password"
    minlength="8" autocomplete="new-password"
    aria-describedby="pwd-hint" required />
  <p id="pwd-hint">Minimum 8 characters.</p>

  <!-- ③ Select dropdown with optgroup -->
  <label for="country">Country</label>
  <select id="country" name="country" autocomplete="country-name">
    <option value="">Select a country</option>
    <optgroup label="North America">
      <option value="US">United States</option>
      <option value="CA">Canada</option>
    </optgroup>
  </select>

  <!-- ④ Textarea -->
  <label for="bio">Bio</label>
  <textarea id="bio" name="bio" rows="4" maxlength="500"
    placeholder="Tell us about yourself..."></textarea>

  <!-- ⑤ Radio group — fieldset + legend required for ally -->
  <fieldset>
    <legend>Preferred contact method</legend>
    <label><input type="radio" name="contact" value="email" checked /> Email</label>
    <label><input type="radio" name="contact" value="phone" /> Phone</label>
  </fieldset>

  <!-- ⑥ Checkbox — wrapping label gives larger click target -->
  <label>
    <input type="checkbox" name="newsletter" value="yes" />
    Subscribe to newsletter
  </label>

  <!-- ⑦ File upload -->
  <label for="avatar">Profile photo</label>
  <input type="file" id="avatar" name="avatar"
    accept="image/png, image/jpeg, image/webp" />

  <!-- ⑧ Hidden field — submitted silently, not shown to user -->
  <input type="hidden" name="csrf_token" value="abc123xyz" />

  <!-- ⑨ Built-in validation attributes -->
  <input type="number" min="1" max="100" />
  <input type="text" pattern="[A-Z]{3}-\d{4}"
    title="Format: ABC-1234" />

  <!-- ⑩ Buttons — always specify type inside a form -->
  <button type="button">Cancel</button> <!-- does NOT submit -->
  <button type="submit">Create Account</button>

</form>
```

TIPS & BEST PRACTICES

- Always use the most specific input type — `type="email"` gives free format validation and the right mobile keyboard
- Add `autocomplete` tokens to every input — enables autofill for users and is a WCAG 13.5 requirement
- Use the wrapping label pattern for checkboxes and radios — larger click target, no `for / id` pairing needed
- Test all forms with keyboard-only navigation — Tab between fields, Space/Enter for checkboxes and submit

QUICK SUMMARY

- Every input needs a `<label>` — `placeholder` is not a substitute
- Group radio/checkbox sets in `<fieldset>` + `<legend>`
- Always specify `type` on `<button>` — default is `submit`
- Use HTML5 input types for free validation, mobile keyboards, and semantic meaning

DESCRIPTION

ARIA (Accessible Rich Internet Applications) is a set of attributes that supplement HTML semantics when native elements aren't sufficient — primarily for custom widgets. The golden rule: **use semantic HTML first**. A native `<button>` beats a `<div role="button">` every time — it's keyboard accessible, correctly announced, and needs no ARIA. Use ARIA only to fill the gaps: communicating dynamic state changes, labelling regions without visible headings, and announcing live updates. ARIA never makes an element keyboard-accessible — you must add that with `tabindex` and event listeners yourself.

NOTES

- Rule of ARIA: no ARIA is better than bad ARIA — incorrect roles actively mislead screen reader users
- `aria-hidden="true"` hides from screen readers but the element remains visible and focusable — never hide interactive elements with it
- `tabindex="0"` adds to natural tab order; `tabindex="-1"` allows programmatic focus only; positive values (avoid) override natural order
- `aria-live="polite"` waits for the user to finish before announcing; `assertive` interrupts immediately — use assertive sparingly
- WCAG 2.1 AA is the most widely required standard — the four principles are Perceivable, Operable, Understandable, Robust (POUR)

COMMON ARIA ROLES

ROLE=	USE WHEN...	NATIVE EQUIVALENT
button	Custom clickable div/span	<button>
checkbox	Custom toggle control	<input type="checkbox">
dialog	Custom modal overlay	<dialog>
alert	Error or important message	No native equivalent
status	Non-urgent live update	No native equivalent
tab	Tab control in a tablist	No native equivalent
tabpanel	Tab panel content area	No native equivalent
tooltip	Hover/focus description	No native equivalent
progressbar	Progress indicator	<progress>

KEY ARIA ATTRIBUTES

ATTRIBUTE	PURPOSE
aria-label	Provides a name when no visible label exists
aria-labelledby	Points to id(s) of visible labelling elements
aria-describedby	Points to id(s) of descriptive/error elements
aria-hidden	Hides decorative elements from screen readers
aria-expanded	true/false — open/closed state of menus, accordions
aria-checked	true/false/mixed — checked state of custom checkboxes
aria-selected	true/false — selected state in listbox, tabs
aria-disabled	true — communicates disabled without removing focusability
aria-invalid	true — marks a field as having a validation error
aria-live	polite / assertive — region announces dynamic changes
aria-atomic	true — read the entire region on any change

FIVE RULES OF ARIA USE

- 1. Use native HTML elements and attributes instead of ARIA whenever possible
- 2. Don't change native semantics unless you really have to
- 3. All interactive ARIA controls must be keyboard usable
- 4. Don't use `role="presentation"` or `aria-hidden="true"` on focusable elements
- 5. All interactive elements must have an accessible name

COMMON MISTAKES

- Adding `role="button"` to a `<div>` without also adding `tabindex="0"` and keyboard handlers — role alone doesn't make it keyboard accessible
- Using `aria-hidden="true"` on a focusable element — screen reader users can still tab to it but get no announcement
- Using `aria-live="assertive"` for all dynamic updates — it interrupts users mid-sentence; reserve for critical, time-sensitive errors
- Updating `aria-expanded` without showing/hiding the controlled content — the state and the visual must match
- Using `aria-label` to override a perfectly good visible label — this creates a mismatch between what's spoken and what's seen

TIPS & BEST PRACTICES

- Test with an actual screen reader (NVDA + Chrome on Windows, VoiceOver + Safari on Mac) — the Accessibility Tree alone isn't enough
- Navigate your page with keyboard only (Tab, Shift+Tab, Enter, Space, Arrow keys) — this catches missing focus, wrong order, and focus traps
- Use the ARIA Authoring Practices Guide (APG) at w3.org/WAI/ARIA/apg for widget patterns — keyboard expectations are documented per role
- Add `:focus-visible` styles — removes the focus ring on mouse click while keeping it for keyboard users

QUICK SUMMARY

- Semantic HTML first — ARIA supplements, never replaces, native elements
- ARIA provides role, name, and state — but never keyboard behavior; add that separately
- `aria-live="polite"` for status updates; `role="alert"` for urgent errors
- Test with keyboard + screen reader — automated tools catch ~30% of ally issues at most

EXAMPLE

```
<!-- ① Native HTML beats ARIA — prefer semantic elements -->
<!-- WRONG: -->
<!-- <div role="button" tabindex="0"
      onclick="save()" onkeydown="handleKey(event)">Save</div> -->
<!-- CORRECT: -->
<button type="button" onclick="save()">Save</button>
<!-- ② aria-label: accessible name when no visible label exists -->
<button aria-label="Close dialog">
  <svg aria-hidden="true" focusable="false"><!-- icon --></svg>
</button>
<!-- ③ aria-labelledby: point to a visible element as the label -->
<section aria-labelledby="pricing-heading">
  <h2 id="pricing-heading">Pricing</h2>
  <!-- section content -->
</section>
<!-- ④ aria-describedby + aria-invalid: field with error -->
<label for="email">Email</label>
<input type="email" id="email"
      aria-describedby="email-error"
      aria-invalid="true" />
<p id="email-error" role="alert">Please enter a valid email address.</p>
<!-- ⑤ aria-expanded: toggle button controlling a panel -->
<button aria-expanded="false" aria-controls="faq-answer" id="faq-q">
  What is your return policy?
</button>
<div id="faq-answer" role="region" aria-labelledby="faq-q" hidden>
  <!-- answer — JS toggles hidden + aria-expanded together -->
</div>
<!-- ⑥ Tab pattern: tablist > tab + tabpanel -->
<div role="tablist" aria-label="Product views">
  <button role="tab" aria-selected="true"
        aria-controls="panel-details" id="tab-details">Details</button>
  <button role="tab" aria-selected="false"
        aria-controls="panel-reviews" id="tab-reviews"
        tabindex="-1">Reviews</button>
</div>
<div role="tabpanel" id="panel-details" aria-labelledby="tab-details">
  <!-- details content -->
</div>
<div role="tabpanel" id="panel-reviews" aria-labelledby="tab-reviews" hidden>
  <!-- reviews content -->
</div>
<!-- ⑦ aria-live: announce dynamic content without a page reload -->
<div aria-live="polite" aria-atomic="true">
  <p id="status-msg"><!-- JS writes "3 results found" here --></p>
</div>
<!-- role="alert" is shorthand for aria-live="assertive" —
      use only for urgent, time-sensitive messages -->
<div role="alert">
  Session expiring in 2 minutes. Save your work.
</div>
<!-- ⑧ Skip link — first focusable element, visually hidden until focused -->
<a href="#main-content" class="skip-link">Skip to main content</a>
<!-- CSS: .skip-link { position:absolute; left:-9999px; }
      .skip-link:focus { left:0; } -->
<!-- ⑨ Focus management: move focus after dialog opens -->
<dialog>
  <!-- tabindex="-1" allows programmatic focus on a non-interactive element -->
  <h2 tabindex="-1">Confirm Action</h2>
  <!-- JS: dialog.showModal(); dialog.querySelector('h2').focus(); -->
  <button>Cancel</button>
  <button>Confirm</button>
</dialog>
```

DESCRIPTION

When a browser parses HTML it builds the **DOM** (Document Object Model) — a live tree of objects representing every element, attribute, and text node. JavaScript interacts with the page by reading and manipulating this tree. This sheet covers the HTML side of that relationship: how `<script>` tags load and execute, how HTML attributes serve as JS hooks (`id`, `class`, `data-*`), and the core DOM methods for selecting and modifying elements. Full JavaScript syntax is covered in the JS reference set — this is the handoff point.

NOTES

- › The DOM is *not* your HTML file — it's a live in-memory tree; JS changes to the DOM don't change the source file
- › `innerHTML` parses HTML strings — never use it with unescaped user input; it's an XSS vector
- › `textContent` sets plain text — safe from XSS; use it when inserting dynamic text
- › `querySelectorAll` returns a static NodeList; `getElementsByClassName` returns a live HTMLCollection
- › Inline event handlers (`onclick="..."`) in HTML are valid but considered bad practice — use `addEventListener` in JS instead

CORE DOM SELECTION METHODS

METHOD	RETURNS	MATCHES
<code>getElementById('id')</code>	Element null	Single element by id
<code>querySelector('css')</code>	Element null	First matching CSS selector
<code>querySelectorAll('css')</code>	NodeList (static)	All matching CSS selectors
<code>getElementsByClassName</code>	HTMLCollection (live)	Elements by class name
<code>getElementsByTagName</code>	HTMLCollection (live)	Elements by tag name
<code>closest('css')</code>	Element null	Nearest ancestor matching selector

READING & WRITING ELEMENTS

PROPERTY / METHOD	PURPOSE
<code>el.textContent</code>	Get/set plain text — XSS safe
<code>el.innerHTML</code>	Get/set HTML — dangerous with user input
<code>el.setAttribute(n, v)</code>	Set any attribute by name
<code>el.getAttribute(n)</code>	Read any attribute by name
<code>el.dataset.key</code>	Read/write <code>data-*</code> attribute (camelCase)
<code>el.classList.add()</code>	Add a class without replacing others
<code>el.classList.toggle()</code>	Add if absent, remove if present
<code>el.style.property</code>	Set inline style — use sparingly
<code>el.remove()</code>	Remove element from the DOM
<code>el.addEventListener()</code>	Attach event handler

ACCESSIBILITY

- › When JS adds or removes content, update ARIA states too — toggling `hidden` on a panel must be paired with updating `aria-expanded`
- › Manage focus after dynamic changes — opening a modal should move focus inside it; closing should return focus to the trigger
- › Use `textContent`, not `innerHTML`, when inserting user-generated text — prevents XSS and keeps the DOM clean
- › Progressive enhancement: build HTML and CSS so the page is usable without JS, then layer JS on top — `<noscript>` is your fallback

COMMON MISTAKES

- › Using `innerHTML` to insert user-provided text — always sanitize or use `textContent`
- › Running DOM queries before the document is parsed — use `defer` on the script or listen for `DOMContentLoaded`
- › Using inline `onclick="..."` handlers in HTML — mixes concerns and can't be removed without editing HTML
- › Selecting elements by tag name when a class or id is more specific — `querySelectorAll('button')` catches far more than you intend
- › Forgetting to update ARIA state when toggling visibility — `hidden` on the panel without updating `aria-expanded` on the trigger

TIPS & BEST PRACTICES

- › Use `data-*` attributes as JS configuration hooks — keeps logic out of class names and is refactor-safe
- › Use `defer` on all external scripts — parallel download, correct execution order, no parser blocking
- › Use event delegation: attach one listener to a parent instead of many listeners to children — more efficient and handles dynamic content
- › Build with progressive enhancement — the page should work without JS for the widest possible audience

QUICK SUMMARY

- › DOM = live tree of HTML in memory; JS reads and writes it at runtime
- › `querySelector` + CSS selectors; `textContent` for safe text; `innerHTML` with caution
- › `defer` for almost all scripts; `data-*` for HTML→JS configuration
- › Full JavaScript syntax continues in the JS reference set — this is the HTML/JS boundary

EXAMPLE - HTML

```
<!-- ① Script loading strategies in <head> -->
<!-- Blocks parsing — only for critical inline scripts -->
<script>
  document.documentElement.classList.add('js');
</script>

<!-- defer: downloads in parallel, executes after DOM is ready -->
<script src="/js/app.js" defer></script>

<!-- async: downloads in parallel, executes immediately on load -->
<script src="/js/analytics.js" async></script>

<!-- ES module: deferred by default, scoped to module -->
<script type="module" src="/js/main.mjs"></script>

<!-- ② HTML hooks that JS targets -->
<button
  id="save-btn"
  class="btn btn--primary"
  data-action="save"
  data-endpoint="/api/save">
  Save
</button>

<div id="status-region" aria-live="polite"></div>
```

EXAMPLE - HTML

```
// ③ Selecting elements
const btn = document.getElementById('save-btn');
const status = document.querySelector('#status-region');
const cards = document.querySelectorAll('.card'); // NodeList

// ④ Reading data-* attributes (HTML: data-endpoint="/api/save")
const endpoint = btn.dataset.endpoint; // "/api/save"
const action = btn.dataset.action; // "save"

// ⑤ Modifying content — textContent is XSS-safe
status.textContent = 'Saving...'; // plain text — safe
status.innerHTML = '<strong>Done!</strong>'; // HTML — risky

// ⑥ Toggling classes and attributes
btn.classList.add('loading');
btn.classList.toggle('active');
btn.setAttribute('aria-busy', 'true');
btn.setAttribute('disabled', '');

// ⑦ Event listeners — preferred over inline onclick
btn.addEventListener('click', async () => {
  btn.classList.add('loading');
  btn.setAttribute('aria-busy', 'true');

  const res = await fetch(btn.dataset.endpoint,
    { method: 'POST' });

  btn.classList.remove('loading');
  btn.removeAttribute('aria-busy');
  status.textContent = res.ok ? 'Saved!' : 'Error saving.';
});

// ⑧ Creating and inserting elements
const card = document.createElement('div');
card.className = 'card';
card.textContent = 'New card content';
document.querySelector('.grid').appendChild(card);

// ⑨ Removing an element
document.querySelector('#old-banner').remove();
```